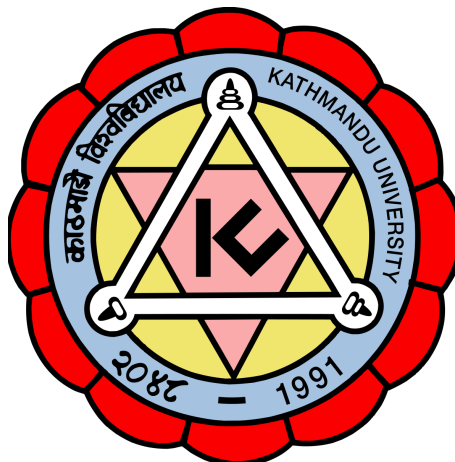


Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



Lab Report 5
[Code No: COMP 342]

Submitted by:

Nischal Subedi (53)

Group: CS60 (III/I)

Submitted to:

Mr. Dhiraj Shrestha

Department of Computer Science and Engineering

Submission Date: 08/02/2026

Q1. Implement the following 3D transformations using the 3D shapes provided by OpenGL:

a. Translation

Translation moves a 3D object from one position to another along the X, Y, and Z axes without changing its size or orientation. It only changes the object's location in space. In PyOpenGL, this is implemented using `glTranslatef(tx, ty, tz)`.

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

# Constants
CUBE_VERTS = [(-1,-1,-1), (1,-1,-1), (1,1,-1), (-1,1,-1), (-1,-1,1),
(1,-1,1), (1,1,1), (-1,1,1)]
CUBE_EDGES = [(0,1), (1,2), (2,3), (3,0), (4,5), (5,6), (6,7), (7,4),
(0,4), (1,5), (2,6), (3,7)]
TRANSLATION = (2.5, 1.5, 0)

def draw_wire_cube(vertices):
    glBegin(GL_LINES)
    for edge in CUBE_EDGES:
        for vertex in edge:
            glVertex3fv(vertices[vertex])
    glEnd()

def draw_axes():
    glLineWidth(2.0)
    glBegin(GL_LINES)
    glColor3f(1, 0, 0); glVertex3f(-5,0,0); glVertex3f(5,0,0) # X -
Red
    glColor3f(0, 1, 0); glVertex3f(0,-5,0); glVertex3f(0,5,0) # Y -
Green
    glColor3f(0, 0, 1); glVertex3f(0,0,-5); glVertex3f(0,0,5) # Z -
Blue
    glEnd()
    glLineWidth(1.0)
```

```

def display():
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glLoadIdentity()
    glTranslatef(0, 0, -10)

    draw_axes()

    # Original Cube - Cyan
    glColor3f(0.2, 0.7, 1.0)
    draw_wire_cube(CUBE_VERTS)

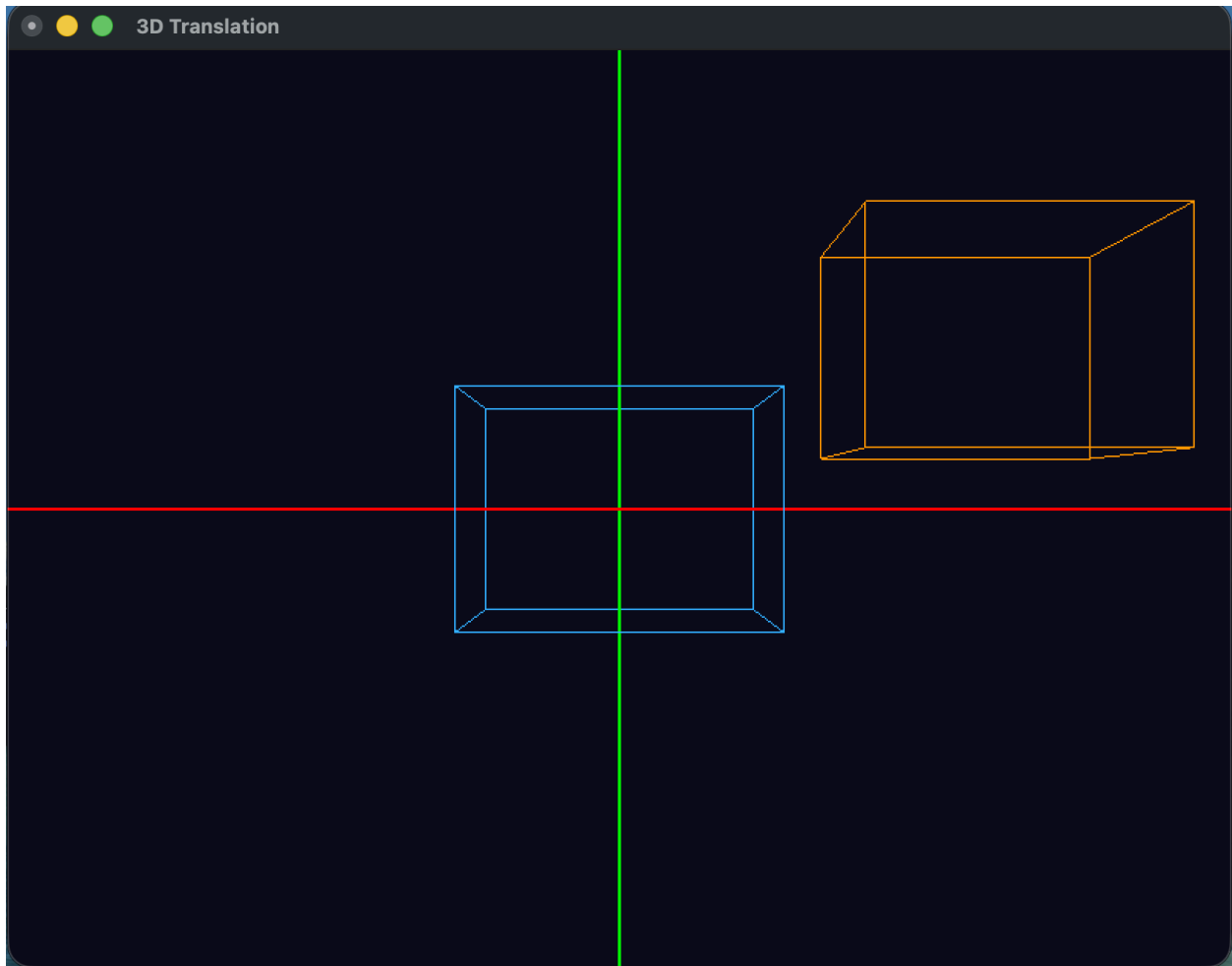
    # Translated Cube - Orange
    translated_verts = [(v[0]+TRANSLATION[0], v[1]+TRANSLATION[1],
v[2]+TRANSLATION[2]) for v in CUBE_VERTS]
    glColor3f(1.0, 0.6, 0.0)
    draw_wire_cube(translated_verts)

    glutSwapBuffers()

def init():
    glClearColor(0.05, 0.05, 0.1, 1)
    glEnable(GL_DEPTH_TEST)
    glMatrixMode(GL_PROJECTION)
    gluPerspective(45, 1, 0.1, 50)
    glMatrixMode(GL_MODELVIEW)

    glutInit()
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH)
    glutInitWindowSize(800, 600)
    glutCreateWindow(b"3D Translation")
    init()
    glutDisplayFunc(display)
    glutMainLoop()

```



b. Rotation

Rotation turns a 3D object around a specified axis (X, Y, or Z). The shape and size remain the same, but the orientation changes. In PyOpenGL, rotation is done using `glRotatef(angle, x, y, z)` where the vector defines the rotation axis.

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

ROT_ANGLE = 45.0
ROT_AXIS = (0, 1, 0)

def display():
```

```

glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
glLoadIdentity()
glTranslatef(0, 0, -7)

# Original - Darker Blue
glColor3f(0.2, 0.5, 0.8)
glutWireCube(2)

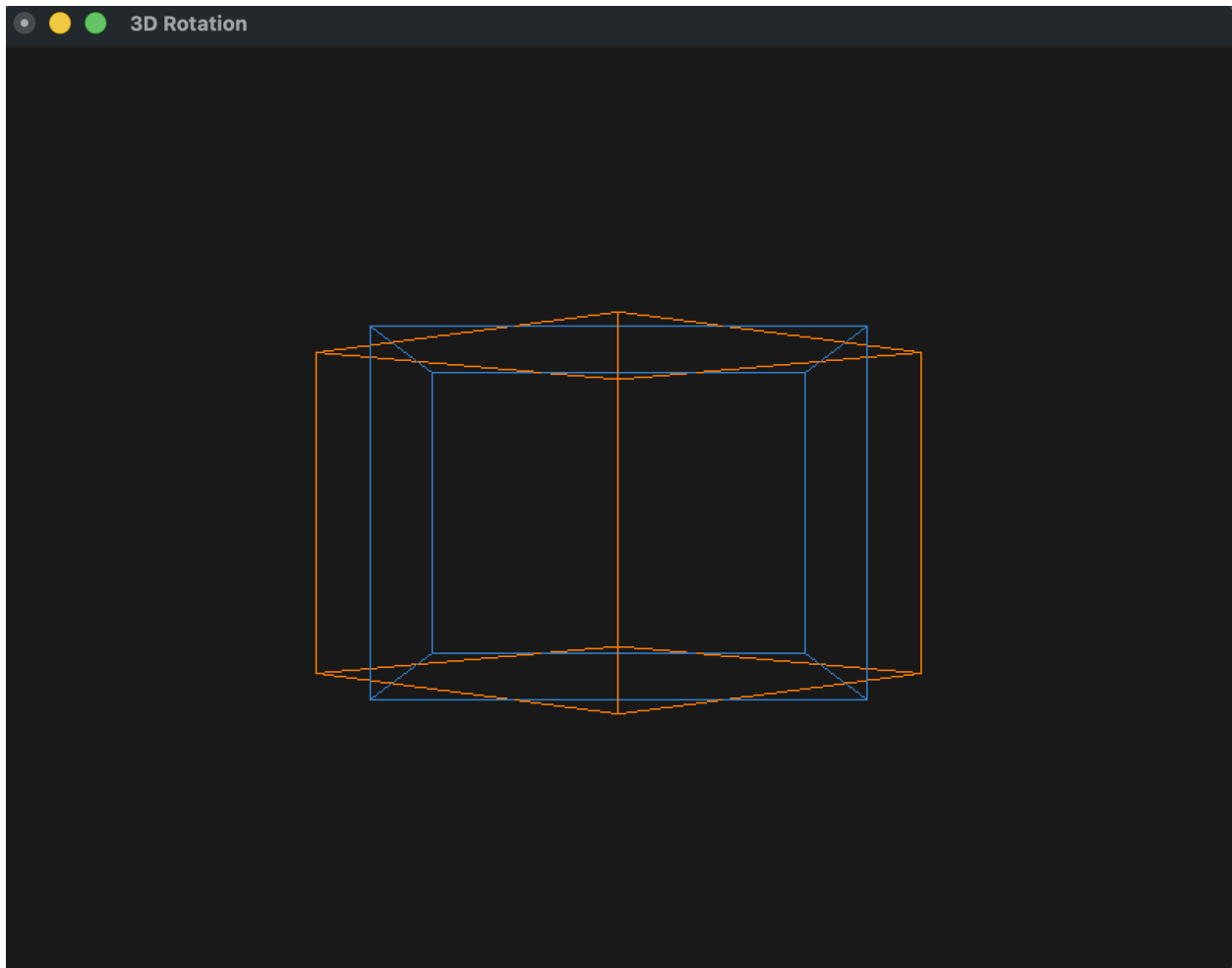
# Rotated - Bright Orange
glPushMatrix()
glRotatef(ROT_ANGLE, *ROT_AXIS)
glColor3f(1.0, 0.5, 0.0)
glutWireCube(2)
glPopMatrix()

glutSwapBuffers()

def init():
    glClearColor(0.1, 0.1, 0.1, 1)
    glEnable(GL_DEPTH_TEST)
    glMatrixMode(GL_PROJECTION)
    gluPerspective(45, 1, 1, 50)
    glMatrixMode(GL_MODELVIEW)

glutInit()
glutInitWindowSize(800, 600)
glutCreateWindow(b"3D Rotation")
init()
glutDisplayFunc(display)
glutMainLoop()

```



c. Scaling

Scaling changes the size of a 3D object by enlarging or shrinking it along the X, Y, and Z directions. It can be uniform or non-uniform. In PyOpenGL, scaling is applied using *glScalef(sx, sy, sz)*.

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

SCALE_FACTOR = (1.5, 0.5, 1.5)

def display():
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glLoadIdentity()
```

```

glTranslatef(0, 0, -8)

# Base Cube - Soft Blue
glColor3f(0.3, 0.6, 0.9)
glutWireCube(2)

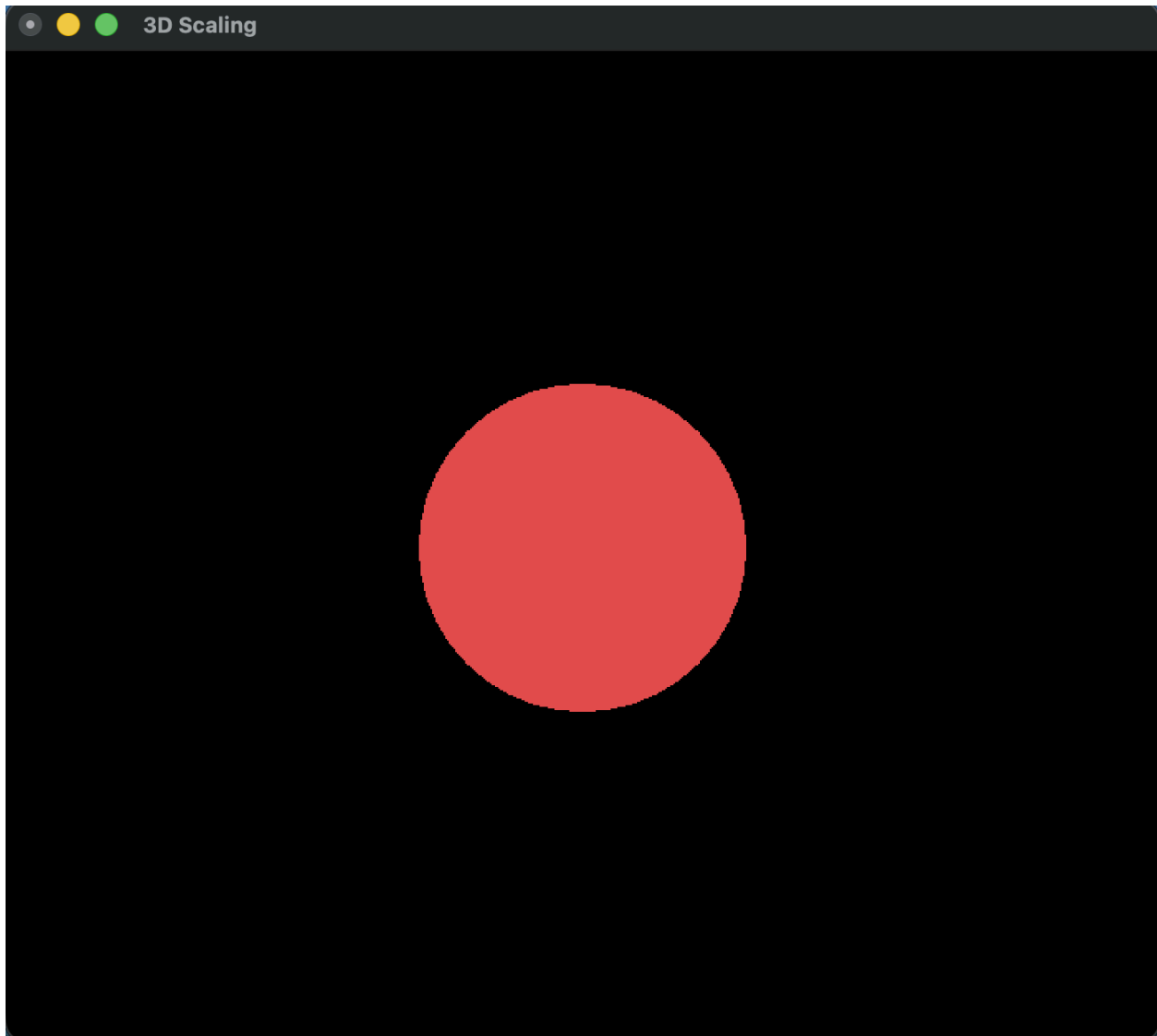
# Scaled Cube - Vivid Amber
glPushMatrix()
glScalef(*SCALE_FACTOR)
glColor3f(1.0, 0.7, 0.1)
glutWireCube(2.01) # Slightly larger to avoid Z-fighting
glPopMatrix()

glutSwapBuffers()

def init():
    glClearColor(0.0, 0.0, 0.0, 1)
    glEnable(GL_DEPTH_TEST)
    glMatrixMode(GL_PROJECTION)
    gluPerspective(45, 1, 1, 50)
    glMatrixMode(GL_MODELVIEW)

glutInit()
glutInitWindowSize(800, 600)
glutCreateWindow(b"3D Scaling")
init()
glutDisplayFunc(display)
glutMainLoop()

```



d. Shearing

Shearing skews or slants the shape of a 3D object by shifting one axis relative to another. OpenGL does not provide a direct shear function, so it is implemented using a custom shear transformation matrix with *glMultMatrixf()*.

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

# X-shear along Y axis
SHEAR_MATRIX = [
```



```

    1, 0.5, 0, 0,
    0, 1, 0, 0,
    0, 0, 1, 0,
    0, 0, 0, 1
]

def display():
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glLoadIdentity()
    glTranslatef(0, 0, -6)

    # Original - Light Blue
    glColor3f(0.4, 0.7, 1.0)
    glutWireCube(1.5)

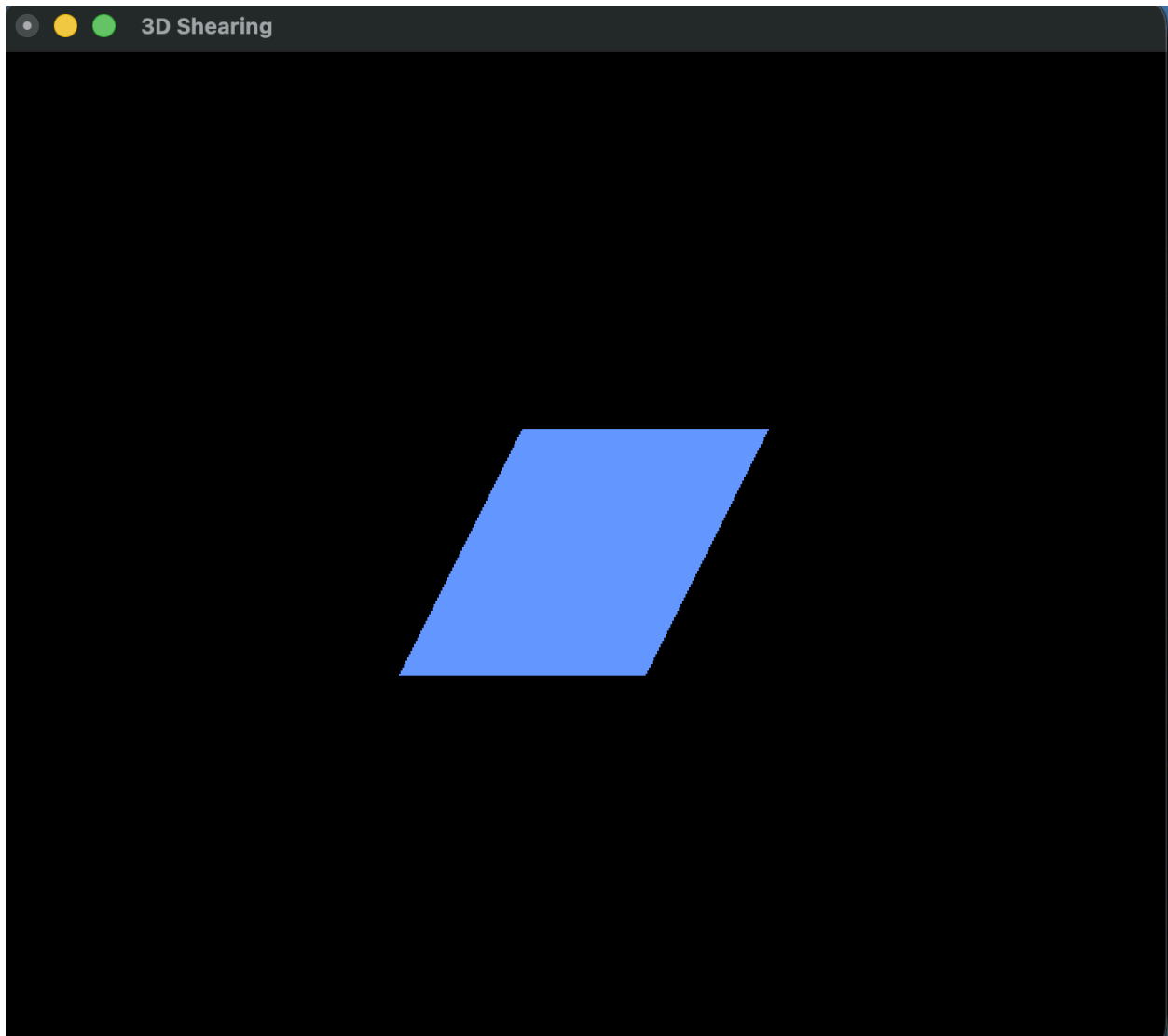
    # Sheared - Gold
    glColor3f(1.0, 0.8, 0.2)
    glPushMatrix()
    glMultMatrixf(SHEAR_MATRIX)
    glutWireCube(1.5)
    glPopMatrix()

    glutSwapBuffers()

def init():
    glClearColor(0.1, 0.1, 0.15, 1)
    glEnable(GL_DEPTH_TEST)
    glMatrixMode(GL_PROJECTION)
    gluPerspective(45, 1, 1, 50)
    glMatrixMode(GL_MODELVIEW)

glutInit()
glutInitWindowSize(800, 600)
glutCreateWindow(b"3D Shearing")
init()
glutDisplayFunc(display)
glutMainLoop()

```



Q2. Implement the Perspective Projection:

Perspective projection displays 3D objects so that distant objects appear smaller and closer objects appear larger, creating realistic depth perception. In PyOpenGL, it is implemented using `gluPerspective(fov, aspect, near, far)`.

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

NEAR_POS = (0, 0, -4)
```

```

FAR_POS = (0, 0, -12)

def draw_grid():
    glColor3f(0.3, 0.3, 0.3)
    glBegin(GL_LINES)
    for i in range(-10, 11):
        glVertex3f(i, -1, -20); glVertex3f(i, -1, 0)
        glVertex3f(-10, -1, -i-10); glVertex3f(10, -1, -i-10)
    glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glLoadIdentity()
    gluLookAt(0, 3, 8, 0, 0, -5, 0, 1, 0)

    draw_grid()

    # Near Cube - Blue
    glPushMatrix()
    glTranslatef(*NEAR_POS)
    glColor3f(0.2, 0.6, 1.0)
    glutWireCube(2)
    glPopMatrix()

    # Far Cube - Orange
    glPushMatrix()
    glTranslatef(*FAR_POS)
    glColor3f(1.0, 0.5, 0.0)
    glutWireCube(2)
    glPopMatrix()

    glutSwapBuffers()

def init():
    glClearColor(0.05, 0.05, 0.05, 1)
    glEnable(GL_DEPTH_TEST)
    glMatrixMode(GL_PROJECTION)
    gluPerspective(60, 1, 1, 100)
    glMatrixMode(GL_MODELVIEW)

```

```
glutInit()  
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH)  
glutInitWindowSize(800, 600)  
glutCreateWindow(b"Perspective Projection")  
init()  
glutDisplayFunc(display)  
glutMainLoop()
```

